

# GIT

Git es una herramienta donde puedes tener un **historial de versiones** de un proyecto. Esto quiere decir que puedes ver el código anterior.

Github es una página web donde amazonas los proyectos en la nube.

En git se trabaja con **ramas**.

Imagina un árbol. El tronco es la rama principal, y ahí vive el código que ya funciona bien. A esa rama se le llama **main**.

Como no queremos arriesgarnos a dañar el tronco, casi nunca trabajamos directamente en él. Para eso existe otra rama llamada **develop**. Piensa en ella como un espacio de pruebas: ahí se juntan los cambios nuevos, y solo cuando todo está bien, se pasan a main.

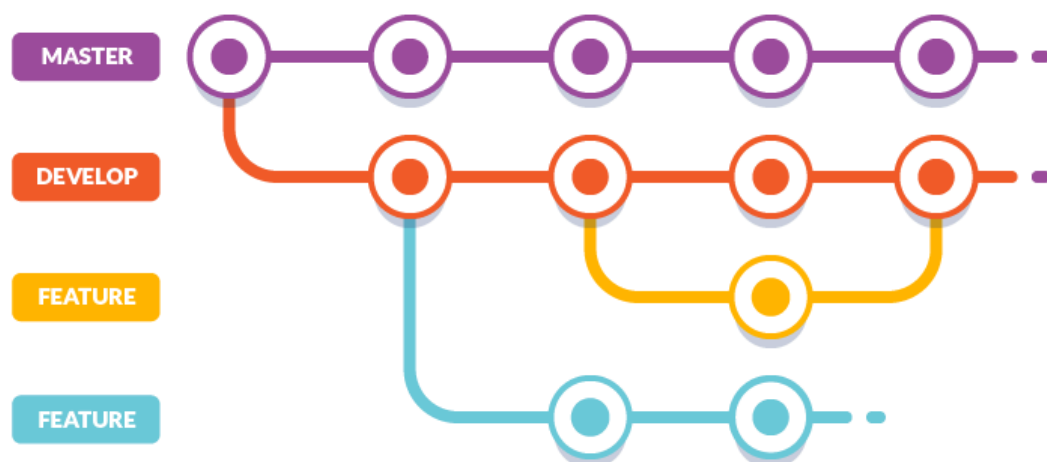
Cuando quieres hacer algo nuevo, por ejemplo agregar un inicio de sesión, no lo haces directo en develop. Creas una rama pequeña que sale de develop, solo para ese cambio. A estas ramas les ponemos un nombre que empieza con **feature/** (que en inglés significa "característica") seguido de lo que vas a hacer. Por ejemplo:

`feature/inicio-sesion`

Así, con solo ver el nombre, cualquiera sabe que en esa rama se está trabajando en el inicio de sesión.

Cuando terminas tu cambio y funciona, esa rama se une de regreso a develop. Y cuando develop está lista y probada, se une a main.

## Imagen de ejemplo



Los puntos que se ven ahí se llaman **commits**.  
Un **commit** es una confirmación de un código que se guardo.

Antes de iniciar a ver los comandos es necesario saber que significa un repositorio local y un repositorio remoto.

## Repositorios

¿Que es un repositorio?

Un repositorio es una carpeta donde se almacenan todos los archivos donde se encuentran los códigos.

Sabiendo que es un repositorio existen dos tipos de repositorios:

1. **Repositorio Local:** Este repositorio es el que almacenas en tu computadora, **Nota:** Todo lo que guardas en este repositorio, solo se almacena en tu computadora.
2. **Repositorio Remoto:** Este repositorio es el que se almacena en la nube, que en este caso es **github**, lugar donde se almacenan todos los archivos que necesitas que estén en la nube, puedes compararlo como si fuera google drive solo que en versión para desarrolladores.

# Comandos de GIT

Existen varios comandos en Git pero los más importantes son:

Esta es una lista de los más utilizados a los que son importantes pero no se utilizan mucho:

## Comandos más utilizados:

1. **git clone:** Clona un repositorio remoto a tu directorio local
2. **git branch:** Este comando tiene dos funciones y son las siguientes
  - a. **Ver ramas:** Este comando sirve para ver las ramas
  - b. **Crear una rama:** Este comando sirve para crear ramas

**Ejemplo:**  
**git branch develop**  
(Con este comando se creó la rama develop)  
**Nota:** Cuando creas esta rama, solo se crea pero no te diriges hacia ella.
3. **git switch:** Es comando sirve para cambiarse de rama  
**Ejemplo:** Quieres cambiarte de la rama develop a la rama main  
**git switch main**
4. **git checkout -b:** Este comando sirve para crear una rama e ir hacia ella
5. **git merge:** Este comando sirve para unir dos ramas. (Quiere decir que los archivos o códigos que tienes en una rama pasan a la rama a la que necesitas los códigos)
6. **git push:** Este comando sirve para enviar los commits al repositorio remoto
7. **git pull:** Este comando sirve para recibir lo nuevo que está en el repositorio remoto y actualiza el repositorio local
8. **git push -u origin:** Este comando sirve para enviar todo los commits locales al repositorio remoto. Este comando se utiliza para subir por primera vez la rama

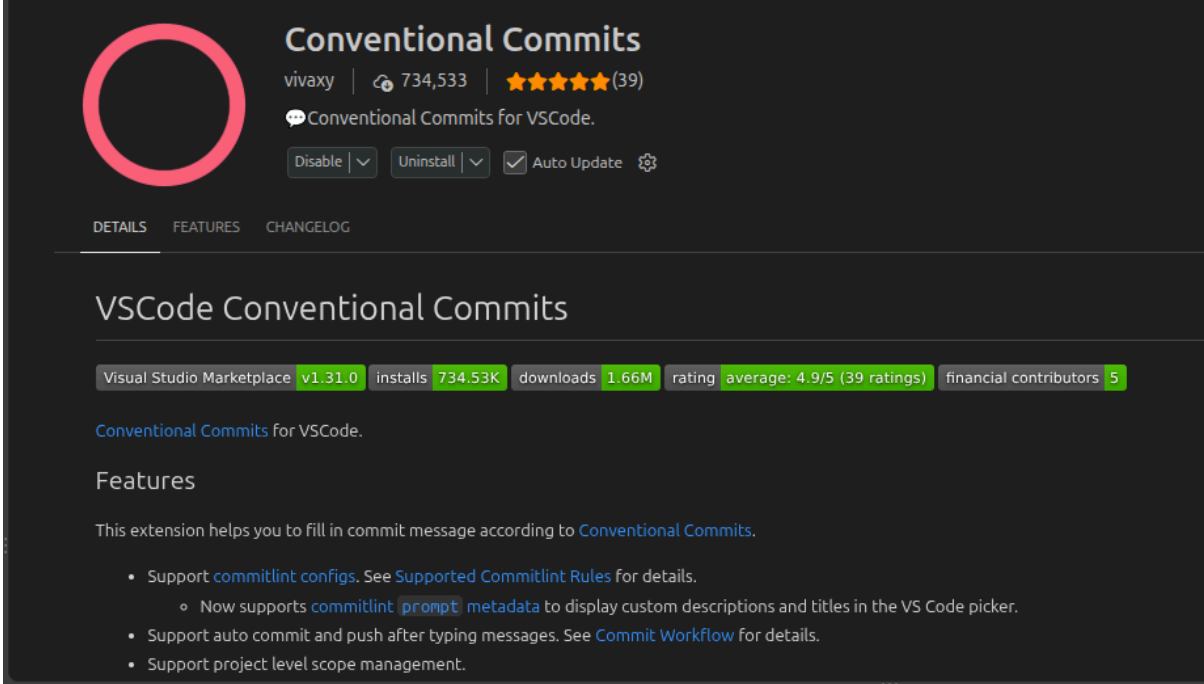
## Comandos menos utilizados:

1. **git status:** Este comando sirve para ver los archivos que son modificados o creados.
2. **git add . :** Este comando sirve para agregar todos los archivos al stage listos para hacer commit
3. **git commit -m:** Este comando sirve para hacer el commit de los archivos que están en el stage. El commit es una confirmación del código que se guardará.
4. **git branch -d:** Este comando sirve para eliminar una rama
5. **git log --oneline:** Este comando sirve para ver en una lista todos los commits en una sola línea junto a la explicación de cada commit.

**Sugerencia:** Para los menos utilizados como el git status, add y el commit no se utilizan mucho ya que nosotros utilizamos la extensión **conventional commit** y esta herramienta nos ayuda hacer estos tres pasos.

## Extensiones:

## Conventional Commit



**Conventional Commits**  
vivaxy | 734,533 | ★★★★★ (39)  
Conventional Commits for VSCode.

Disable | Uninstall | Auto Update

DETAILS | FEATURES | CHANGELOG

### VSCode Conventional Commits

Visual Studio Marketplace | v1.31.0 | installs 734.53K | downloads 1.66M | rating average: 4.9/5 (39 ratings) | financial contributors 5

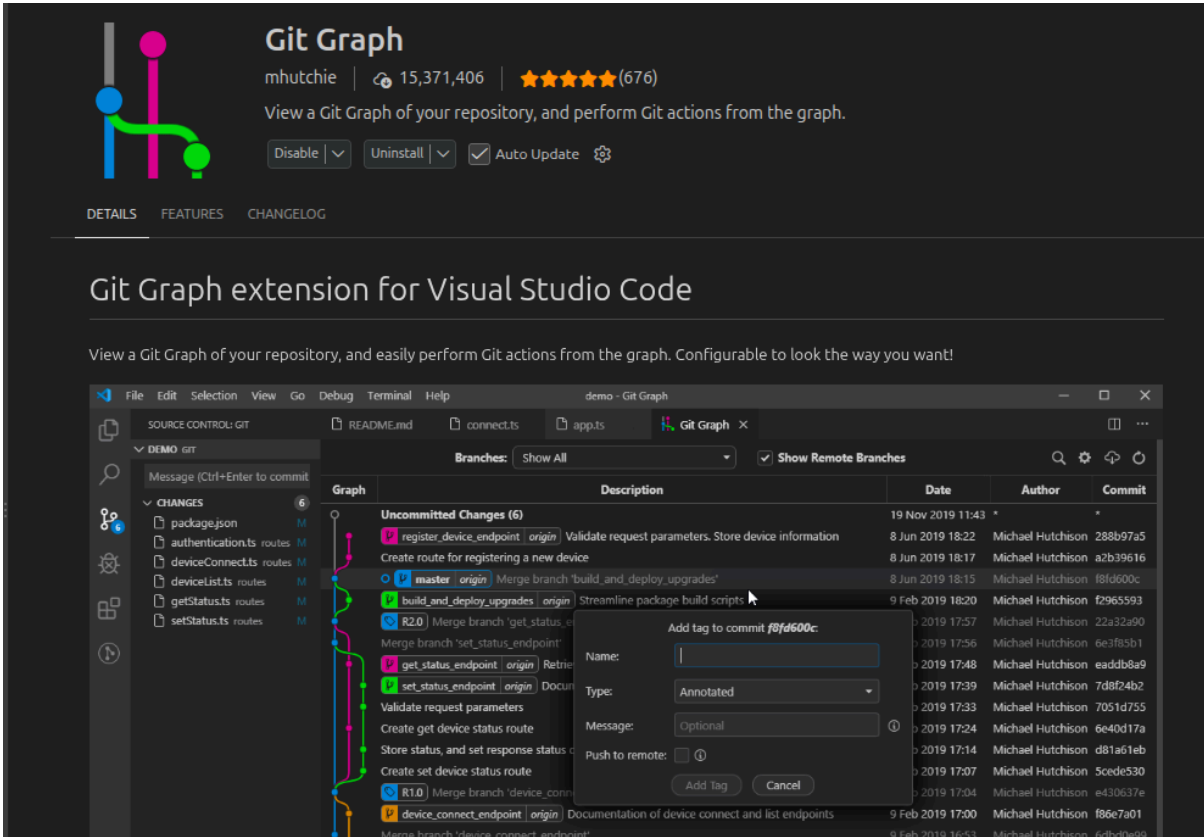
Conventional Commits for VSCode.

#### Features

This extension helps you to fill in commit message according to [Conventional Commits](#).

- Support [commitlint configs](#). See [Supported Commitlint Rules](#) for details.
  - Now supports [commitlint prompt](#) [metadata](#) to display custom descriptions and titles in the VS Code picker.
- Support auto commit and push after typing messages. See [Commit Workflow](#) for details.
- Support project level scope management.

## Git graph: Extensión para ver las ramas.



**Git Graph**  
mhutchie | 15,371,406 | ★★★★★ (676)  
View a Git Graph of your repository, and perform Git actions from the graph.

Disable | Uninstall | Auto Update

DETAILS | FEATURES | CHANGELOG

### Git Graph extension for Visual Studio Code

View a Git Graph of your repository, and easily perform Git actions from the graph. Configurable to look the way you want!

demo - Git Graph

Source Control: GIT | README.md | connect.ts | app.ts | Git Graph

Branches: Show All | Show Remote Branches

Message (Ctrl+Enter to commit)

CHANGES

- package.json
- authentication.ts routes
- deviceConnect.ts routes
- deviceList.ts routes
- getStatus.ts routes
- setStatus.ts routes

Graph

|                                  | Description                                           | Date              | Author            | Commit   |
|----------------------------------|-------------------------------------------------------|-------------------|-------------------|----------|
| Uncommitted Changes (6)          |                                                       | 19 Nov 2019 11:43 | *                 | *        |
| register_device_endpoint origin  | Validate request parameters. Store device information | 8 Jun 2019 18:22  | Michael Hutchison | 288b97a5 |
|                                  | Create route for registering a new device             | 8 Jun 2019 18:17  | Michael Hutchison | a2b39616 |
| master origin                    | Merge branch 'build_and_deploy_upgrades'              | 8 Jun 2019 18:15  | Michael Hutchison | f8d600c  |
| build_and_deploy_upgrades origin | Streamline package build scripts                      | 9 Feb 2019 18:20  | Michael Hutchison | f2965593 |
| R2.0                             | Merge branch 'get_status_endpoint'                    | 2019 17:57        | Michael Hutchison | 22a32a90 |
|                                  | Merge branch 'set_status_endpoint'                    | 2019 17:56        | Michael Hutchison | 6e3f85b1 |
| get_status_endpoint origin       | Retrieve device status                                | 2019 17:48        | Michael Hutchison | eaddb8a9 |
|                                  | Validate request parameters                           | 2019 17:39        | Michael Hutchison | 7d8f24b2 |
|                                  | Create get device status route                        | 2019 17:33        | Michael Hutchison | 7051d755 |
|                                  | Store status, and set response status                 | 2019 17:24        | Michael Hutchison | 6e40d17a |
|                                  | Create set device status route                        | 2019 17:14        | Michael Hutchison | d81a61eb |
| R1.0                             | Merge branch 'device_connect_endpoint'                | 2019 17:07        | Michael Hutchison | Sced530  |
|                                  | Documentation of device connect and list endpoints    | 9 Feb 2019 17:00  | Michael Hutchison | R86e7a01 |
| device_connect_endpoint origin   |                                                       | 9 Feb 2019 16:53  | Michael Hutchison | 6dbd0e99 |
|                                  | Merge branch 'device_connect_endpoint'                |                   |                   |          |

Add tag to commit f8fd600c:

Name:

Type: Annotated

Message: Optional

Push to remote: ☐

Add Tag Cancel